Docker Networking

Docker Networking in **Docker** controls how containers communicate with each other, the host machine, and external networks.

---

1. Bridge Network

The **bridge network** is the default network Docker creates for containers on one host machine.

- Each container gets its own **internal IP address**.
- Containers can communicate with each other if they are on the same bridge network.
- It acts like a **private internal network**.

Example:

docker run -d --name web nginx
docker run -d --name db mysql

Both containers are connected to the default **bridge network**.

You can see networks with:

docker network ls

---

2. Host Network

In **host network mode**, the container shares the **same network as the host machine**.

- No separate container IP.
- The container uses the host's ports directly.
- Faster networking because there is no network isolation.

Example:

docker run --network host nginx

If **Nginx** runs on port 80, it will use the host machine's port 80.

---

3. Container Communication

Containers can communicate with each other in several ways.

Using Container Name

If containers are on the same network, they can connect using the container name as the hostname.

Example:

Backend container connects to database container:

mysql://db:3306

Here `db` is the container name running **MySQL**.

Example:

```
docker network create mynetwork
docker run -d --name db --network mynetwork mysql
docker run -d --name app --network mynetwork myapp
```

Now `app` can connect to `db`.

---

4. Port Mapping

Containers run inside isolated networks. To allow users to access them from outside, **port mapping** is used.

Example:

```
docker run -p 8080:80 nginx
```

Meaning:

```
Host Port → Container Port
8080      → 80
```

When a user visits:

http://localhost:8080

The request goes to port **80 inside the container** where **Nginx** is running.

Simple Real Example (Web + Database)

Application architecture:

User → Web Container → Database Container

- Web server: **Nginx**
- Database: **MySQL**

Networking flow:

1. User connects through mapped port.
2. Web container receives request.
3. Web container communicates with database container using container name.

Quick Summary

Concept	Purpose
Bridge Network	Default private network for containers
Host Network	Container uses host's network
Container Communication	Containers talk using name/IP
Port Mapping	Expose container service to outside

What is the difference 1. Bridge Network vs 3. Container Communication

Bridge Network vs Container Communication

1. Bridge Network

A **bridge network** is a **type of network created by Docker**. It connects multiple containers on the same host machine so they can exist in the same virtual network.

Key points:

- Default Docker network.
- Containers receive **private internal IP addresses**.
- Acts like a **virtual LAN** inside the host.
- Enables containers to be in the same network environment.

Example:

docker network create mynetwork

---

2. Container Communication

Container communication refers to **how containers send data to each other** when they are connected to a network (such as a bridge network).

Methods:

- Using **container name**
- Using **container IP**
- Using **service name** (with tools like **Docker Compose**)

Example:

mysql://db:3306

Here:

- `db` = database container name running **MySQL**
- Another container connects to it.

Simple Analogy

Concept	Analogy
Bridge Network	The **road system** connecting houses
Container Communication	The **cars traveling between houses**

Simple Summary

Feature	Bridge Network	Container Communication
What it is	Network type	Data exchange between containers
Purpose	Connect containers to the same network	Allow containers to talk to each other
Role	Infrastructure	Usage of that infrastructure

Bridge network **creates the environment**, while container communication **uses that environment to send requests between containers**.

mysql://db:3306 how to work with command can explain this command means

`mysql://db:3306` is **not a Docker command**. It is a **connection string** used by an application to connect to a **MySQL** database running in another container in **Docker**.

Structure of the Connection String

mysql://db:3306

Parts:

Part	Meaning
`mysql://`	Database protocol (MySQL connection)
`db`	Hostname (the database container name)
`3306`	MySQL default port

So it means:

“Connect to the MySQL server running in container `db` on port 3306.”

Example with Docker Containers

Create a network:

docker network create mynetwork

Run MySQL container:

```
docker run -d \
--name db \
--network mynetwork \
-e MYSQL_ROOT_PASSWORD=1234 \
mysql
```

Run an application container:

```
docker run -d \
--name app \
--network mynetwork \
myapp
```

How Communication Works

Inside the **app container**, the application can connect to the database using:

```
mysql://db:3306
```

Explanation:

- `db` → Docker automatically resolves this to the **IP address of the db container**.
- `3306` → MySQL server port.
- The request goes through the **Docker network**.

Flow:

App Container → db:3306 → MySQL Container

Full Real Connection Example

A full connection string usually looks like:

```
mysql://username:password@db:3306/mydatabase
```

Example:

```
mysql://root:1234@db:3306/hotel_db
```

Meaning:

- user = root
- password = 1234
- database = hotel_db
- server = db container

Why the Name `db` Works

In a Docker network:

- Container names act like **DNS hostnames**.
- Docker automatically resolves them.

So `db` becomes something like:

172.18.0.2

But you do not need to know the IP.

If needed, the **Docker Compose example (web app + MySQL)** can also be explained, because this connection string is commonly used there.